AtomQuest Hackathon 1.0 Grand Finale

Real-Time Video Support Platform — Login Credentials & Architecture

Submitted by: Rudransh Dwivedi

1. Access & Login Credentials

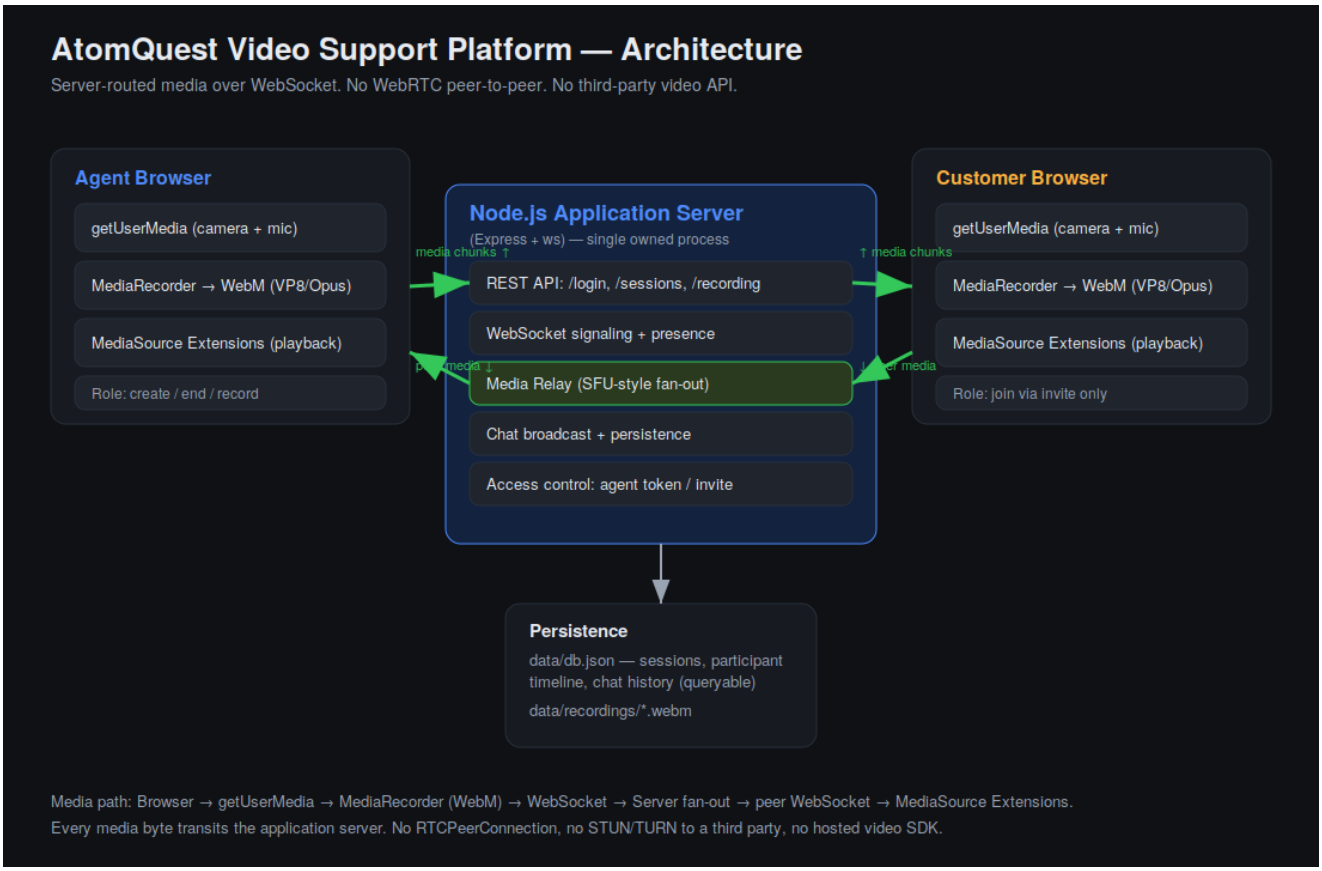
The platform runs locally after `npm install` then `npm start`, served at `http://localhost:3000` (open in Chrome). There are two roles. Judges can switch between them as follows.

Role	How to access	Capabilities
Call Agent	Sign in on the home page. Username: agent Password: agent123	Create sessions, generate invite links, start/stop recording, en
Customer	Open the invite link the agent generates (.../room.html?session=ID&invite=TOKEN), enter a name, click Join.	Join a single session via a valid invite. Cannot create or end s

A second seeded agent (`agent2 / agent123`) is also available. To demo a full call, sign in as the agent in one tab and open the invite link in a second tab or incognito window.

2. System Architecture

Media is routed entirely through the application server. There is no WebRTC peer-to-peer connection and no third-party hosted video API. Each browser captures its camera and mic with getUserMedia, encodes the stream with MediaRecorder (WebM, VP8/Opus), and sends the chunks over a WebSocket to the Node server, which fans them out to the other participant for playback via Media Source Extensions.



3. Component summary

- **REST API** (Express): agent login, session create/list/detail, end session, recording download.
- **WebSocket layer** (ws): join/presence signaling, binary media fan-out, real-time chat, mute/video state.
- **Access control**: agent bearer token vs per-session invite token, validated on every join.
- **Persistence**: JSON session store (participants, timeline, chat history) plus recording files on disk.